Dong Joo Kwon
Professor Chan
DATA_ENG 300-0
19 April 2026

Homework 1: Aircraft Inventory Analysis

Part 1:

I began part 1 of the assignment by examining the extent of the missing values across the columns of interest. I found that there were 49, 105, 3, 13238, 13332, and 105 missing entries for CARRIER, CARRIER_NAME, MANUFACTURE_YEAR, NUMBER_OF_SEATS, CAPACITY_IN_POUNDS, and AIRLINE_ID, respectively.

For each of CARRIER, CARRIER_NAME, MANUFACTURE_YEAR, and AIRLINE_ID, my general approach was to print the affected rows and inspect what other values were present alongside them. This would allow me to identify any relationships among the existing data and the build mappings to fill in the missing entries.

CARRIER: After inspecting the missing rows, I noticed that what was really happening was that Pandas was automatically interpreting the string "NA" as a null value when it actually stood for "North American Airlines". These are not truly missing values. So, I filled these entries in with the string "NA" to restore the correct values.

CARRIER_NAME: The missing values in this column correspond exactly to the rows where CARRIER was "NA", confirming the same root cause. I filled those rows with "North American Airlines", as needed.

AIRLINE_ID: For this column, I constructed a lookup dictionary from rows within inventory that already had both the CARRIER and AIRLINE_ID values populated. Then, I used each row's carrier code to map the correct AIRLINE_ID back to the rows where it was missing.

MANUFACTURE_YEAR: Since this column had only three missing values, I used Simple Median Imputation. Median imputation was more appropriate than mean here because mean would produce fractional years, which are not meaningful in this context. Since there were such few missing values, I believed that filling all the missing entries with the same values would not be an issue.

For both NUMBER_OF_SEATS and CAPACITY_IN_POUNDS, I first analyzed the type of missingness. I created indicator columns NUMBER_SEATS_MISSING and CAPACITY_MISSING with binary values (1 = missing, 0 = not missing), then calculated the proportion of missing values for each AIRCRAFT_TYPE group. The results showed that certain aircraft types had proportions of exactly 0.0 or 1.0, meaning entire aircraft types were either fully

missing or fully present. If the data were Missing Completely at Random (MCAR), we would expect missingness to be spread relatively evenly across all aircraft types. Instead, the pattern shows that missingness is systematically tied to aircraft type, indicating the data is **Missing at Random (MAR)** where the probability of a value being missing is dependent on other observed variables (in this case, AIRCRAFT_TYPE), but not on the missing value itself. MNAR would imply the missingness is related to the value itself, but since the missingness pattern aligns with AIRCRAFT_TYPE groups rather than the magnitude of the values, MAR is ultimately the more appropriate classification.

To fill in the missing data, I considered the four imputation methods we learned about in class. KNN and PMM were ruled out due to the size of the dataset. Both methods are computationally expensive as they require calculating distances or fitting models across all rows, which felt impractical at this scale.

<u>NUMBER_OF_SEATS and CAPACITY_IN_POUNDS</u>: For both of these columns, I used a grouped median imputation strategy, filling missing values with the median of their respective AIRCRAFT_TYPEs. My reasoning for choosing this strategy is that aircraft of the same type are physically similar machines, and will therefore have comparable seating capacities and load limits. This makes the group median a much more meaningful estimate than a global one across all aircraft types which would happen if I used the Simple Median Imputation strategy. Simple Median Imputation would also ignore group structure and produce a less meaningful estimate, whereas grouped median balances accuracy and computational feasibility at this dataset size. Median was preferred over mean for the same reason as with MANUFACTURE_YEAR; the distribution of values for both columns could be subject to skewness due to outliers that are unusually large or small, so the median provided a more robust central estimate.

<u>Part 2:</u>

Before altering any data, I began by exploring each column, filtering to show only the most frequent values (the “big wins”). Specifically, I looked at the manufacturers that appear more than 50 times and the model appearing more than 100 times. This way, I can identify the inconsistencies that are worth addressing to improve the dataset.

<u>AIRCRAFT_STATUS and OPERATING_STATUS</u>: The inconsistencies here lied in there being the same status values with inconsistent whitespace and capitalization. I resolved this by stripping leading and trailing whitespace and converting all values to uppercase. One entry in OPERATING_STATUS was empty; rather than dropping the row and losing potentially useful data for later analysis, I filled it with 'None' to show that the data was not assigned to a status while still preserving the record.

<u>MANUFACTURER</u>: I broke the cleaning into two steps. First, I stripped whitespace and removed all special characters (spaces, hyphens, and periods) from every entry, then converted

everything to uppercase to create a normalized convention for future comparison. Second, I focused on the "big winners" (filtered the manufacturers with more than 50 aircraft in the dataset), since correcting these would have the greatest impact on the dateset. I built a mapping dictionary to consolidate the names, because there were many variants for the same manufacturers, into a single consistent form (for instance, "THE BOEING CO", "BOEING CO", and "THE BOEING COMPANY" all mapped to "BOEING").

MODEL: I applied the same baseline cleaning of stripping whitespace, uppercasing, and collapsing multiple spaces to ensure consistency across entries. As I did with the MANUFACTURER column, the focus of the data correction was on high-frequency values where standardization would have the most effect on future analysis.

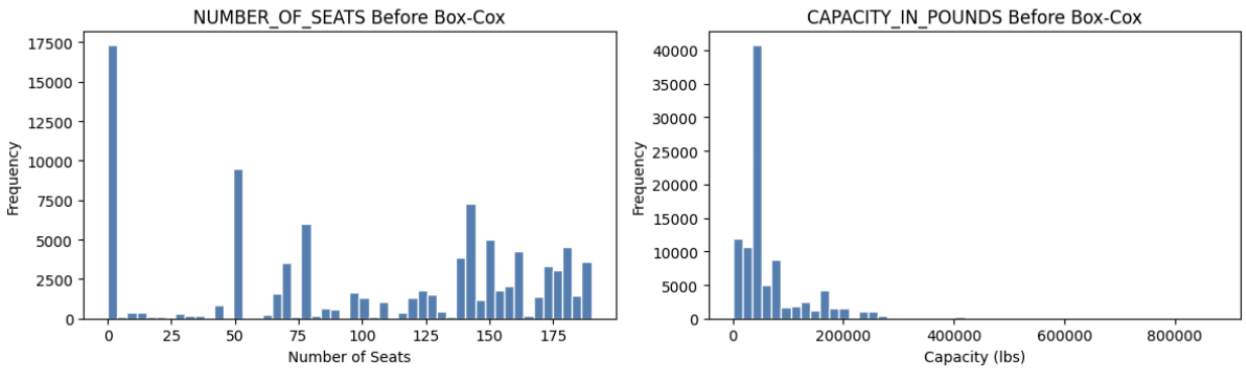
Part 3:

After dropping any rows that still contained missing values, 101275 rows remained. Since the original number of rows was 132313, this means that 76.5% of the data remained.

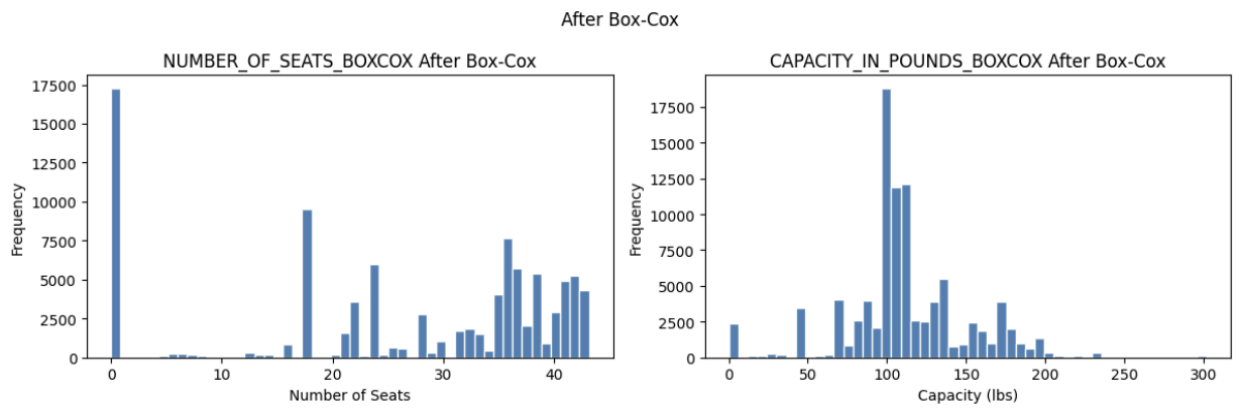
Part 4:

NUMBER_OF_SEATS: Before the Box-Cox transformation, the distribution demonstrated a mild left/negative skewness (skewness = -0.324). In the plot, you can see a pronounced spike near zero and another cluster around 50 seats, reflecting the mix of small and larger aircrafts in the database. The Box-Cox transformation applied only a modest correction, as indicated by the lambda value 0.638 which is close to 1. This is consistent with the already near-normal skewness value. The shape of the distribution after the transformation looks incredibly similar, with the x-axis rescaled but the same general pattern preserved.

CAPACITY_IN_POUNDS: Before the Box-Cox transformation, this column was heavily right-skewed (skewness = 4.052), with a majority of aircraft clustered at low capacity values and a long tail stretching out. This is a pretty common pattern indicating that a normal transformation is needed to get the data to resemble more of a Bell Curve. After applying the Box-Cox transformation (generated a lambda value = 0.339), the distribution is substantially more symmetric and spread out, with values now concentrated around the center of the range rather than piled up at the lower end. The right tail has been handled, the plot now producing a much more bell-shaped distribution that is better suited for our statistical analysis.



Histograms of columns before Box-Cox transformation



Histograms of columns after Box-Cox transformation

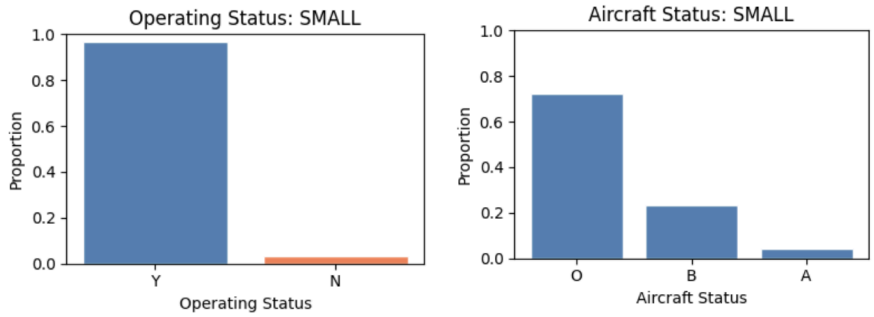
<u>Part 5:</u>

Regarding operating rates, I found that they are consistently high across all size groups, with the vast majority of aircrafts being active regardless of size. The “xlarge” aircrafts have the highest operating rate at 97.3%, followed by “small” at 96.8% and “large” at 96.3%. “Medium” aircraft have the lowest operating rate at 94.4%, though this difference is miniscule compared to the other proportions. Overall, aircraft size does not appear to have a strong relationship with whether an aircraft is currently operating.

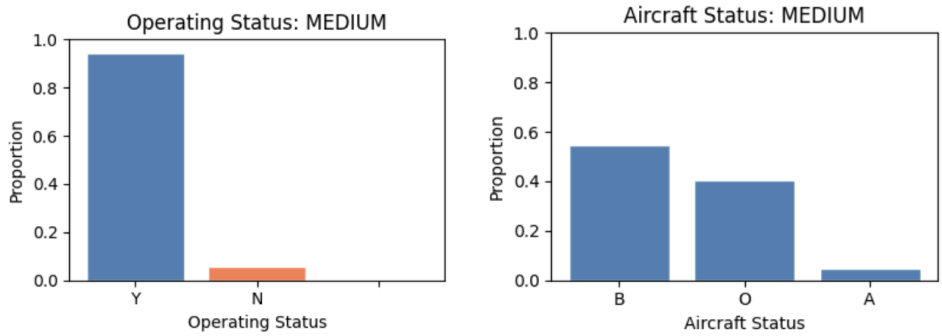
Interestingly, more patterns emerge when looking at aircraft status. "Small" aircrafts are predominantly in the status O (72.3%), with a significant proportion in status B (23.5%). "Medium" aircraft showed a different pattern, where status B was the most common at 54.7%, exceeding status O (40.7%). This suggests that smaller-to-mid-sized aircrafts are more likely to fall into status B.

"Large" and "Xlarge" aircrafts return to a pattern dominated by status O (65.1% and 70.1% respectively), with the rate of status B dropping back down compared to "Medium". Noticeably,

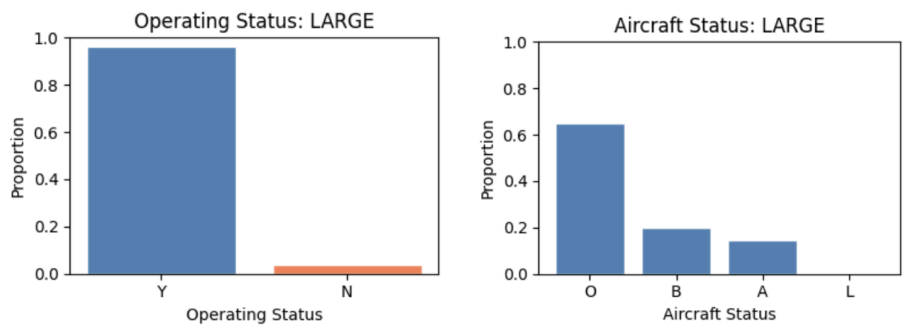
"Large" aircrafts have a higher proportion of status A (14.6%) compared to the other size groups. Status L appears only in "Large" and "Xlarge" groups, though at very small proportions.



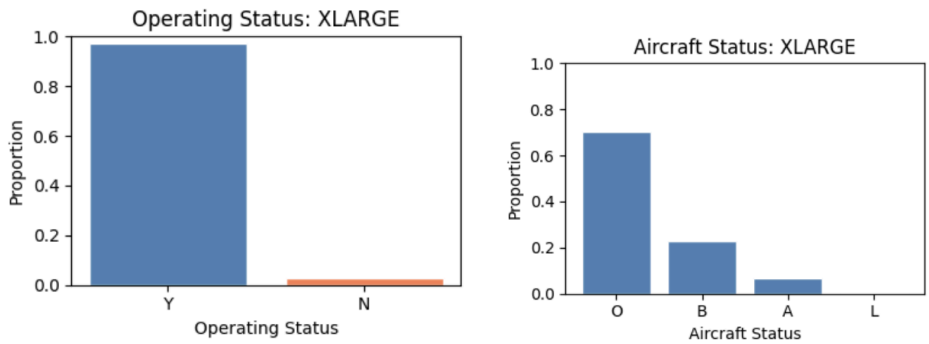
Small: Proportions of Operating Status and Aircraft Status



Medium: Proportions of Operating Status and Aircraft Status



Large: Proportions of Operating Status and Aircraft Status



XLarge: Proportions of Operating Status and Aircraft Status

<u>Part 6:</u>

As the assignment states, I randomly split the cleaned dataset into 80% training and 20% testing sets, then trained four models for NUMBER_OF_SEATS and CAPACITY_IN_POUNDS. To tune the models, I tried multiple values for n_estimators to lower the RMSE values. I ultimately landed on 100 as it is a widely used default that provides stable predictions without creating a lot of computational cost. Increasing the parameter to >100 produced barely noticeable improvements in the RMSE, likely due to additional trees contributing diminishing returns. Thus, 100 trees was sufficient in capturing the data.

The following table holds the RMSE values I generated for each model:

Model	Target Column	RMSE: Training	RMSE: Testing
Linear Regression	NUMBER_OF_SEATS	60.70	61.36
Random Forest	NUMBER_OF_SEATS	3.12	3.32
Linear Regression	CAPACITY_IN_POUNDS	67941	69010
Random Forest	CAPACITY_IN_POUNDS	35558	39537

Based on the RMSE values, it is clear that the random forest model substantially outperformed linear regression for both target columns. For NUMBER_OF_SEATS, the random forest resulted in a test RMSE of 3.32 (seats) compared to 61.36 for linear regression; this is roughly 18 times better. This suggests that the relationship between the features and number of seats is likely non-linear, which is why linear regression could not adequately capture the pattern; however, random forest identified it well through its use of decision trees. The grouped median imputation for NUMBER_OF_SEATS likely contributed to the low variance within AIRCRAFT_TYPE groups, which may have made the random forest's job easier; this provides a bit of explanation behind its strong performance.

A similar pattern holds for CAPACITY_IN_POUNDS, where the random forest's test RMSE of 39,537 lbs was almost half the RMSE for linear regression of 69,010 lbs. The gap is smaller in relative terms than for NUMBER_OF_SEATS, but still very big. In both models, the training and testing RMSEs are close to one another, which indicates that neither model is severely overfitting to the training data and both models generalize reasonably well to unseen data.

AI Usage:

Tool: Claude

Key prompt: For Part 2 (specifically the MANUFACTURER and MODEL standardization/transformation), I provided Claude with my filtered list of high-frequency manufacturer and model names (the "big wins" extracted from the dataset via value_counts()) and asked it to identify naming inconsistencies, variants of the same manufacturer, and errors that should be corrected.

What I changed and how I verified: I designed the mapping dictionary approach independently and used Claude to identify which name variants should map to the same manufacturer. I manually cross-referenced each suggested mapping against the original list I had provided to verify accuracy. For example, I confirmed that every name Claude flagged as “BOEING” was genuinely a Boeing entry. The final mapping dictionary reflects the corrections I independently verified.